



NOSQL

Unit 2

Subject Code: CIAEC49

B.E Sem: IV

Credits : 1

What is MongoDB?

Definition: An open-source, document-oriented NoSQL database designed for high volume data storage.

Key Characteristics:

- **Schema-less:** Documents in the same collection don't need the same set of fields.
- **BSON Format:** Stores data in "Binary JSON," which allows for faster parsing.
- **Scalability:** Built-in horizontal scaling via "Sharding."
- **High Availability:** Uses "Replica Sets" to ensure data is always reachable.



Definition: A document is a set of key-value pairs.

Dynamic Schema: This is the "superpower" of MongoDB.

- Documents in the same collection do not need the same set of fields.
- One document might have 5 fields, while the next has 50.

Data Type Flexibility: Common fields across documents can hold different types of data.

- Example: In document A, "zipcode" could be an Integer (560001). In document B, it could be a String ("BT1 1AA").

Structure: Documents are stored in BSON (Binary JSON) format, which allows for complex nested structures.

```
{
  _id: ObjectId("7df78ad8902c"),
  title: "MongoDB Overview",
  description: "MongoDB is a NoSQL database",
  by: "tutorials point",
  url: "http://www.tutorialspoint.com",
  tags: ["mongodb", "database", "NoSQL"],
  likes: 100,
  comments: [
    {
      user: "user1",
      message: "My first comment",
      dateCreated: new Date(2011,1,20,2,15),
      like: 0
    }
  ]
}
```

The Collection

- **Definition:** A collection is a grouping of MongoDB documents.
- **The RDBMS Equivalent:** It serves as the counterpart to a Table.
- **Hierarchy:** A collection exists within a single database.
- **The "No-Schema" Rule:** Collections do not enforce a structure. You don't need to "Alter Table" to add a new field.



The CRUD Cycle

C – Create: Inserting new documents into a collection.

R – Read: Querying documents from a collection.

U – Update: Modifying existing documents.

D – Delete: Removing documents from a collection.



MongoDB Data Types

Null: Represents a null value or a non-existent field. {"x" : null}

Boolean: Stores true or false. {"active" : true}

String: UTF-8 encoded characters. {"name" : "MongoDB"}

Number: The shell defaults to 64-bit floating point (e.g., {"x": 3.14}).

For exact integers, use:

- NumberInt("3"): 4-byte signed integer.
- NumberLong("3"): 8-byte signed integer.



MongoDB Data Types

Array: Stores lists or sets of values (can even store nested documents).

Object (Embedded Document): A document inside a document.

ObjectId: Small, nearly unique 12-byte identifier for the _id field.

Date: 64-bit integer representing milliseconds since Unix epoch.



The use Command

- MongoDB **use DATABASE_NAME** is used to create database.
- The command will create a new database if it doesn't exist, otherwise it will return the existing database

```
use DATABASE_NAME
```

```
test> use mydb
```

```
switched to db mydb
```

If you want to check your databases list, use the command **show dbs**.

```
mydb>show dbs  
  
admin      40.00 KiB  
config    100.00 KiB  
local      40.00 KiB
```



The dropDatabase() Method

MongoDB db.dropDatabase() command is used to drop a existing database.

```
db.dropDatabase()
```

The createCollection() Method

- MongoDB db.createCollection(name, options) is used to create collection.

```
db.createCollection(name, options)
```

```
mydb>use test  
switched to db test  
>db.createCollection("myCollection")  
{ ok : 1 }
```

The drop() Method

MongoDB's `db.collection.drop()` is used to drop a collection from the database.

```
db.COLLECTION_NAME.drop()
```

The "Create" (Insert)

What is it? Adding new documents to a collection.

The Methods:

`db.collection.insertOne( { ... } )`: Adds a single document.

`db.collection.insertMany([ { ... }, { ... } ])`: Adds multiple documents via an array.

The `_id` Field: If you don't provide an `_id`, MongoDB generates a unique ObjectId automatically.



```
test> db.empDetails.insertOne(
...   {
...     First_Name: "Radhika",
...     Last_Name: "Sharma",
...     Date_Of_Birth: "1995-09-26",
...     e_mail:
"radhika_sharma.123@gmail.com",
...     phone: "9848022338"
...   })
{
  acknowledged: true,
  insertedId: ObjectId('6964b6ee25c4ea6b911e2625')
}
```

```
db.users.insertMany([
  {
    name: "Bob",
    age: 30,
    hobbies: ["gaming", "cooking"],
    status: "active"
  },
  {
    name: "Charlie",
    age: 22,
    hobbies: ["photography"],
    status: "pending"
  },
  {
    name: "Diana",
    age: 28,
    hobbies: ["yoga", "hiking"],
    status: "active"
  }
])
```

The "Read" Operation – Finding Data

- **Method:** `db.collection.find(query, projection)`
- **The find() Method:** Returns a "cursor" to the matching documents.
- **The findOne() Method:** Returns the first document that matches the criteria.
- **Empty Query:** `db.users.find({})` returns all documents in the collection.
- **Pretty Print:** Adding `.pretty()` at the end makes the JSON output readable in the shell.

1. The `find()` Method

Returns a cursor (an iterator) to all documents matching the criteria.

- **Example (Find all):** `db.users.find({})`
- **Example (Find with filter):** `db.users.find({ age: 25 })`
- **Example (Find with Projection):** `db.users.find({ age: 25 }, { name: 1, _id: 0 })`
 - *Result:* Returns only the names of 25-year-olds.